

SESIÓN 17 y 18 – INSERCIÓN, MODIFICACIÓN Y BORRADO DE REGISTROS (DML)

Objetivo de la sesión

- Entender **qué es SQL** y para qué sirve.
- Diferenciar **DDL** y **DML**.
- Insertar datos con **INSERT**.
- Modificar datos con **UPDATE**.
- Eliminar datos con **DELETE**.
- Trabajar con **XAMPP + phpMyAdmin**.

1. ¿Qué es SQL?

SQL (Structured Query Language) es el idioma que usamos para **hablar con una base de datos**.

Comparación:

- Excel: haces clics
- Base de datos: escribes SQL

Con SQL podemos:

- Crear tablas
- Guardar datos
- Cambiar datos
- Borrar datos
- Consultar datos

Nos centramos en **DML = trabajar con los datos**.

2. DDL vs DML

Lenguaje	¿Para qué sirve?	Ejemplos
DDL	Crear la estructura	CREATE TABLE
DML	Manipular los datos	INSERT, UPDATE, DELETE

3. Entorno de trabajo: XAMPP + phpMyAdmin

3.1 Arrancar XAMPP

1. Abrir **XAMPP Control Panel**
2. Iniciar:
 - Apache
 - MySQL

3.2 Entrar en phpMyAdmin

- Navegador: <http://localhost/phpmyadmin>

Aquí:

- Vemos las bases de datos
- Escribimos SQL
- Vemos tablas y registros

4. Base de datos de trabajo

Vamos a trabajar con una base muy sencilla: **Tienda**.

4.1 Crear base de datos

```
CREATE DATABASE tienda;  
USE tienda;
```

4.2 Crear tabla Clientes

```
CREATE TABLE clientes (  
  id_cliente INT PRIMARY KEY,  
  nombre VARCHAR(50),  
  correo VARCHAR(100),  
  edad INT  
);
```

A partir de aquí **SOLO DML**.

5. INSERT – Insertar datos

5.1 ¿Qué hace INSERT?

Añade filas nuevas a una tabla.

- Una **fila** = un registro (una “persona”, un “producto”, una “reserva”...)
- Una **columna** = un dato concreto (nombre, correo, edad...)

Si la tabla está vacía, INSERT es lo primero que usas.

5.2 Sintaxis básica

```
INSERT INTO tabla (col1, col2, col3)
VALUES (valor1, valor2, valor3);
```

Cómo se lee:

- **INSERT INTO clientes:** voy a meter datos en la tabla clientes.
- **(id_cliente, nombre, correo, edad):** en estas columnas exactas.
- **VALUES (...):** estos son los valores que voy a guardar.

Reglas clave (para evitar errores):

- El **orden de los valores** debe coincidir con el **orden de columnas**.
- Los textos van entre **comillas simples** '...'.
Ejemplo: nombre = 'Ana', correo = 'ana@email.com'
- Los números van **sin comillas**.
Ejemplo: edad = 22
- Cada instrucción termina con ;.

5.3 Primer INSERT

```
INSERT INTO clientes (id_cliente, nombre, correo, edad)
VALUES (1, 'Ana', 'ana@email.com', 22);
```

Qué está pasando aquí:

- id_cliente = 1 : identificador único (no se debe repetir)
- nombre = 'Ana' : texto con comillas
- correo = 'ana@email.com' : texto con comillas
- edad = 22 : número

Siempre comprobamos que se guardó:

```
SELECT * FROM clientes;
```

Cómo se lee:

- “selecciona todo (*) de clientes” y me muestra todas las filas y columnas.

5.4 Insertar varios registros de golpe

```
INSERT INTO clientes (id_cliente, nombre, correo, edad)
VALUES
(2, 'Luis', 'luis@email.com', 25),
(3, 'Marta', 'marta@email.com', 19),
(4, 'Carlos', 'carlos@email.com', 30);
```

Novedades:

- Con una sola orden, insertas varias filas.
- Cada fila va entre paréntesis.
- Se separan con comas.
- El último termina con ;.

Comprobación:

```
SELECT * FROM clientes;
```

5.5 Insertar SOLO algunas columnas

A veces no queremos rellenar todo.

Ejemplo: insertamos un cliente sin correo (por ahora):

```
INSERT INTO clientes (id_cliente, nombre, edad)
VALUES (5, 'Sofía', 28);
```

Qué ocurre con correo:

- Se queda en **NULL** (vacío) porque no le dimos valor.

Comprobación:

```
SELECT * FROM clientes WHERE id_cliente = 5;
```

Cómo se lee:

- muéstrame el cliente cuyo id_cliente sea 5.

5.6 Insertar con error típico

ERROR: texto sin comillas

```
INSERT INTO clientes (id_cliente, nombre, correo, edad)
VALUES (6, Ana, ana@email.com, 20);
```

Por qué falla:

- Ana y ana@email.com son texto → necesitan '...'

Corrección:

```
INSERT INTO clientes (id_cliente, nombre, correo, edad)
VALUES (6, 'Ana', 'ana@email.com', 20);
```

5.7 Error típico 2: repetir la clave primaria

Si ya existe id_cliente = 1, no puedes volver a usarlo.

```
INSERT INTO clientes (id_cliente, nombre, correo, edad)
VALUES (1, 'Otra Ana', 'otra@email.com', 40);
```

Por qué falla:

- id_cliente es **PRIMARY KEY** : no se repite.

Solución: usa otro id

```
INSERT INTO clientes (id_cliente, nombre, correo, edad)
VALUES (7, 'Otra Ana', 'otra@email.com', 40);
```

5.8 Checklist de INSERT

Antes de ejecutar:

- ¿He puesto comillas en los textos?
- ¿Los valores están en el mismo orden que las columnas?
- ¿El id no está repetido?
- ¿Termina con ;?

6. UPDATE – Modificar datos

6.1 ¿Qué hace UPDATE?

Cambia valores que ya existen.

IMPORTANTE:

- UPDATE **no crea filas nuevas**.
- UPDATE **edita** filas existentes.

PELIGRO REAL: si no usas WHERE, modificas **toda la tabla**.

6.2 Sintaxis

UPDATE tabla
SET columna = valor
WHERE condicion;

Cómo se lee:

- **UPDATE clientes** : voy a modificar la tabla clientes.
- **SET edad = 23** : la edad pasa a valer 23.
- **WHERE id_cliente = 1** : solo del cliente 1

6.3 Cambiar un dato concreto

Queremos que Ana (id 1) tenga 23 años.

1. Primero miramos la fila (buena práctica):
SELECT * FROM clientes WHERE id_cliente = 1;

2. Ahora modificamos:
UPDATE clientes
SET edad = 23
WHERE id_cliente = 1;

3. Comprobamos:
SELECT * FROM clientes WHERE id_cliente = 1;

6.4 Cambiar varios campos a la vez

```
UPDATE clientes  
SET correo = 'ana_nuevo@email.com', edad = 24  
WHERE id_cliente = 1;
```

Qué está pasando:

- Con una sola instrucción, cambias dos columnas.
- Se separan por coma dentro de SET.

Comprobación:

```
SELECT * FROM clientes WHERE id_cliente = 1;
```

6.5 UPDATE con operación matemática

Subimos 1 año a quien tenga más de 20.

1. Primero vemos quiénes serían afectados:

```
SELECT * FROM clientes WHERE edad > 20;
```

2. Ahora actualizamos:

```
UPDATE clientes  
SET edad = edad + 1  
WHERE edad > 20;
```

Cómo se lee:

- edad pasa a ser edad + 1 : incrementa, no fija.

3. Comprobamos:

```
SELECT * FROM clientes;
```

6.6 EL GRAN ERROR (sin WHERE)

```
UPDATE clientes  
SET edad = 99;
```

Qué hace:

- Cambia la edad a 99 **de todos los clientes**.

Regla de oro:

Si no hay WHERE, afecta a toda la tabla.

6.7 Checklist de UPDATE

Antes de ejecutar:

- ¿He hecho un SELECT con el mismo WHERE?
- ¿Tengo WHERE?
- ¿Estoy cambiando el dato correcto?
- ¿Necesito cambiar 1 fila o muchas?

7. DELETE – Borrar registros

7.1 ¿Qué hace DELETE?

Elimina filas completas (registros enteros).

Importante:

- No borra “una columna”, borra **la fila entera**.
- Si borras un cliente, desaparece todo lo suyo (id, nombre, correo, edad).

Si no usas WHERE, borras **todo**.

7.2 Sintaxis

DELETE FROM tabla

WHERE condicion;

Cómo se lee:

- **DELETE FROM clientes** : voy a borrar de la tabla clientes
- **WHERE id_cliente = 3** : solo al cliente con id 3

7.3 Borrar un registro concreto

Queremos borrar a Marta (id 3).

1. Primero vemos a quién borraríamos:

```
SELECT * FROM clientes WHERE id_cliente = 3;
```

2. Si es correcto, borramos:

```
DELETE FROM clientes
```

```
WHERE id_cliente = 3;
```


3. Comprobamos que ya no está:
`SELECT * FROM clientes WHERE id_cliente = 3;`

7.4 Borrar por condición (varias filas)

Borrar clientes menores de 21.

1. Verificación:
`SELECT * FROM clientes WHERE edad < 21;`

2. Borrado:
`DELETE FROM clientes
WHERE edad < 21;`

3. Comprobación:
`SELECT * FROM clientes;`

7.5 EL ERROR MÁS GRAVE (sin WHERE)

`DELETE FROM clientes;`

Qué hace:

- Borra **todas las filas** de la tabla.

Regla de oro:
`DELETE` sin `WHERE` = tabla vacía.

7.6 Checklist de DELETE

Antes de ejecutar:

- ¿He hecho un `SELECT` con el mismo `WHERE`?
- ¿Estoy borrando lo que quiero?
- ¿Es 1 fila o muchas?
- ¿Tengo copia/entorno de pruebas?

8. Buenas prácticas PROFESIONALES

8.1 Siempre comprobar antes

Ejemplo: si quiero borrar menores de 21...

Primero:

`SELECT * FROM clientes WHERE edad < 21;`

Después:

DELETE FROM clientes WHERE edad < 21;

8.2 Orden correcto

1. **SELECT** (ver qué voy a tocar)
2. **UPDATE o DELETE** (hacer el cambio)
3. **SELECT** (comprobar)

10. Resumen

- **INSERT** : añade filas
- **UPDATE** : modifica filas
- **DELETE** : elimina filas
- **WHERE** : **salva vidas**
- **SELECT** : comprueba siempre

SQL no es difícil: **es exacto**.

Si escribes exactamente lo que quieres, hace exactamente eso.